Session Checkpoint

Streamlit Supervisor Dashboard

Real-Time Anomaly Detection Platform

July 21, 2026

Contents

1 Session Objectives

Design and implement the Streamlit supervisor dashboard — the only user-facing component of the anomaly detection platform. The dashboard provides alert investigation, KPI analytics, operation simulation, and pipeline health monitoring for branch supervisors.

2 Architecture Decisions

2.1 Data Flow Pattern

The dashboard follows a strict client pattern with separated read and write paths:

- **Reads:** Dashboard → PostgreSQL (direct connection pool, cached queries)
- **Writes:** Dashboard → FastAPI (`/simulate`, `/feedback`)
- **Metrics:** Dashboard → Prometheus HTTP API

Rationale: Writes go through FastAPI to maintain a single source of truth for mutations, enable validation, and support downstream side effects (e.g., feedback loop to `evaluation_labels`). Reads bypass FastAPI to avoid building 6–8 read endpoints under deadline pressure — read queries are safe, idempotent, and cacheable.

2.2 Caching Strategy

- `@st.cache_resource` for the PostgreSQL connection (singleton, session lifetime)
- `@st.cache_data(ttl=10)` for alert queries (10-second freshness)
- `@st.cache_data(ttl=30)` for KPI aggregations (30-second freshness)
- `streamlit-autorefresh` at 10-second interval replaces manual refresh button

Rationale: The dashboard is a review and investigation tool, not the first responder. The streaming pipeline and decision service are the real-time path. A 10-second cache TTL provides near-current data while preventing PostgreSQL hits on every Streamlit re-run. Auto-refresh eliminates the need for a manual refresh button and provides the supervisor with a continuously updated view.

2.3 Connection Resilience

All database queries route through a `safe_query()` wrapper that catches connection failures, clears the cached connection via `st.cache_resource.clear()`, and retries with a fresh connection. One retry on failure; if both fail, the exception bubbles up naturally.

2.4 Multi-Page Structure

Streamlit multi-page apps isolate each page's execution scope — navigating to the KPIs page only executes `2_kpis.py`, not the alerts query. This provides inherently lower latency for navigation compared to a single-page approach with conditional blocks.

Sidebar content (pipeline status, pending alert counts) is extracted into a shared `components.py` module and called from every page to ensure persistence across navigation.

2.5 Supervisor Feedback via API

Supervisor confirm/reject decisions are submitted as `POST /feedback` to the FastAPI simulation API rather than writing directly to PostgreSQL from the dashboard. The endpoint:

1. Updates `alerts.supervisor_decision` and `alerts.decision_timestamp`
2. Inserts into `evaluation_labels` with `ON CONFLICT` upsert (supports decision changes)

This positions the feedback loop for the retraining pipeline — supervisor decisions flow through a single write path.

3 Dashboard Pages

3.1 Sidebar (All Pages)

- **Pipeline status indicator:** Queries Prometheus `up{job=~'enrichment|scoring|decision'}` and displays green/red status. Graceful degradation to "Prometheus unreachable" warning.
- **Pending alert counts:** Grouped by tier (BLOCK/ALERT/REVIEW), color-coded. Refreshes every 10 seconds with autorefresh.

3.2 Page 1 — Alerts (List + Detail + Feedback)

- **Filters:** Tier, operation type, supervisor decision status (pending/confirmed/rejected/all)
- **Alert list:** Color-coded expandable rows with tier badge, operation type, amount, timestamp, score, status
- **Alert detail (expanded):** Client/employee/branch IDs, score, tier, operation type. Triggered rules from `reasons` column. Regulatory flags with warning icons. SHAP explanation: human-readable NLG sentences from `explanation.reasons`, with expandable raw AE feature contributions and LSTM deviations from `explanation.raw`.
- **Supervisor feedback:** Confirm/reject buttons with optional comment. POST to `/feedback`, then `st.cache_data.clear()` + `st.rerun()` for immediate UI refresh.

3.3 Page 2 — KPIs

- **Top-level metrics:** Total alerts, BLOCK count, ALERT count, REVIEW count
- **Charts (Plotly):** Pie chart by tier distribution, bar chart by operation type, line chart of alerts per day, bar chart of top 20 branches by alert count
- **TTL:** 30 seconds for all aggregation queries

3.4 Page 3 — Simulation

- **Inputs:** Client ID (dropdown of known clients from alerts table, or manual entry), Employee ID (same pattern), amount, operation type, conditional payload fields (beneficiary ID for virement, emitter ID for cheque)
- **Validation:** Client-side checks before API call (required fields, positive amount, payload consistency)
- **Output:** Tier badge (color-coded), fused score, triggered rules, regulatory flags, expandable AE/LSTM SHAP explanations

3.5 Page 4 — Pipeline Health

- **Service status:** Green/red indicators for all 6 Prometheus-scraped services (enrichment, scoring, decision, raw-events-sink, decisions-sink, kafka-exporter)
- **Throughput:** Events/minute per service via `rate(events_processed_total[1m])`
- **Latency:** P95 processing duration in milliseconds via `histogram_quantile(0.95, ...)`
- **Errors:** Error counts over last 5 minutes, grouped by service and error class
- **Consumer lag:** Per consumer group/topic/partition from kafka exporter, with warning threshold at 100 messages sustained lag

4 API Extension — Feedback Endpoint

```
POST /api/v1/feedback

Request:
{
    "event_id": "uuid-string",
    "decision": "confirmed" | "rejected",
    "comment": "optional string"
}

Response:
{
    "status": "ok",
    "event_id": "uuid-string",
    "decision": "confirmed"
}
```

- PostgreSQL connection pool (`psycopg2.pool.SimpleConnectionPool`) added to FastAPI lifespan alongside Redis and model dependencies
- Synchronous `psycopg2` chosen over async `asyncpg` — supervisor feedback is low-frequency (a few actions per hour per branch), and the entire codebase is synchronous
- `ON CONFLICT` upsert on `evaluation_labels` allows supervisors to change decisions
- 404 returned if `event_id` not found in alerts table

5 Docker Integration

- **Dockerfile.dashboard:** Python 3.11-slim, minimal dependencies (`requirements-dashboard.txt`)
- **Docker Compose service:** Port 8501, depends on `postgres` healthy. Environment variables for DB, API URL (via Docker service names), and Prometheus URL.
- **Service count:** 18 total Docker Compose services (17 previous + dashboard)
- **Resource note:** Full stack (18 services) is resource-intensive. Minimum viable set for dashboard testing: postgres, redis, redpanda, simulation-api, hydrate-redis, prometheus, kafka-exporter, dashboard (8 services)

6 Bug Fixes

- **prometheus.yml corruption:** File contained docker-compose service definitions instead of Prometheus scrape configuration. Replaced with correct `scrape_configs` block. Detected via Claude Code diagnostic.
- **Missing `dotenv` loading:** Local PostgreSQL password not read from `.env` file. Added `load_dotenv()` to `db.py` and `app.py`.
- **Sidebar persistence:** Sidebar content disappeared on page navigation because each page runs independently. Extracted into shared `components.py` called from every page.
- **Schema mismatch (local):** Local PostgreSQL still had old schema (`severity_tier`, `anomaly_score`). Resolved by testing in Docker where `init.sql` had created the correct schema.

7 Files Created / Modified

File	Action
src/dashboard/app.py	Created (entry point, page config, autorefresh)
src/dashboard/db.py	Created (connection, safe_query, cached queries)
src/dashboard/components.py	Created (shared sidebar rendering)
src/dashboard/pages/1_alerts.py	Created (alert list, detail, SHAP, feedback)
src/dashboard/pages/2_kpis.py	Created (metrics cards, Plotly charts)
src/dashboard/pages/3_simulation.py	Created (form, API call, result display)
src/dashboard/pages/4_health.py	Created (Prometheus queries, 5 sections)
src/api/models.py	Modified (added FeedbackRequest)
src/api/routes.py	Modified (added POST /feedback, imports)
src/api/app.py	Modified (added pg_pool to lifespan)
Dockerfile.dashboard	Created
requirements-dashboard.txt	Created
docker-compose.yml	Modified (added dashboard service, total 18)
config/prometheus.yml	Fixed (replaced docker-compose content with scrape config)

8 Design Considerations for Production

- **Push notifications:** Current architecture uses 10-second polling via autorefresh. Production systems would add a push mechanism (WebSocket, AlertManager webhook, SMS) for time-critical BLOCK alerts. Streamlit does not support native WebSocket push; `streamlit-autorefresh` provides 90% of the UX with minimal implementation cost.
- **Read API layer:** In a multi-team environment, dashboard reads would go through dedicated API endpoints to decouple frontend from schema changes. Deferred due to timeline — 6–8 additional endpoints not justified for single-developer project.
- **Branch-level filtering:** Global branch filter in sidebar would enable per-branch views. Not implemented as the system operates at aggregated level for demo purposes.

9 Updated Roadmap (Priority Order)

1. **Model retraining** — AE + LSTM on 30-feature schema (fixes inflated scores across dashboard)
2. **Dead letter queue** — route failed events to DLQ topic instead of silent drops
3. **Structured JSON logging** — replace print statements across all services
4. **MLflow** — wire into weekly_retrain DAG placeholder
5. **Rapport de stage** — 30–40 pages, assembled from weekly reports and checkpoints
6. **Soutenance preparation** — slides, demo script, live walkthrough

Deadline: July 28, 2026 — 7 days remaining.